

The GLP–Graph Book Authoring Pipeline

Hybrid Intelligence Evolving Beyond the Monkeys with Typewriters

Joachim De Witte

September 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY–NC–ND 4.0 license.

Abstract

The GLP–Graph Book Authoring Pipeline demonstrates how a compact recursive attractor system can generate long-form narrative structure without relying on transformer-style sequence modeling. In this work, GLPG is reinterpreted as a deterministic transition map: each lemma selects its own attractor state, and generation unfolds as a canon-compliant trajectory through a learned world model. This architecture enables hybrid intelligence in which structural, temporal, and contextual signals modulate an autonomous narrative motor rather than drive it.

A central contribution of this paper is the systematic improvement of the GLPG input vector. We introduce a multi-component encoding layer consisting of a normalized line fraction, a high-resolution current sentence representation, multi-scale token histories, and a lightweight narrative memory. These signals enrich the attractor dynamics without increasing model complexity, allowing GLPG to maintain coherence, pacing, and structural segmentation across hundreds of generated chapters.

We further refine the training regime through sentence-level sampling, multi-book curricula, structural token annotation, and balanced epoch selection. The generative engine is extended with a fully parametrizable `WriterParams` layer, enabling explicit control over chapter structure, temporal steering, attractor routing, beam dynamics, and token-level modulation. Optional learned activation and bias functions are explored as modular extensions to the recursive core.

Together, these components show that long-form narrative generation can emerge from a small, deterministic dynamical system when equipped with rich contextual encodings and structurally informed training. GLPG illustrates a path toward hybrid intelligence: compact attractor architectures augmented by modular steering layers, capable of producing coherent books under strict structural constraints while remaining fully interpretable and parametrizable.

1 Introduction

Hybrid intelligence systems challenge the assumption that long-form narrative generation requires large-scale probabilistic models. The GLP–Graph Book Authoring Pipeline (GLPG) demonstrates that coherent books, complete with chapters, paragraphs, pacing, and canon, can emerge from a compact deterministic architecture built around attractor dynamics rather than transformer-style sequence modeling. In GLPG, every lemma corresponds to a stable attractor state, and generation unfolds as a trajectory through a learned transition map. This perspective reframes narrative generation as the evolution of a dynamical system rather than the sampling of a token distribution.

The original GLPG architecture relied on paging to cluster lemmas into shared attractor regions. In this work, we adopt a simplified configuration in which each lemma occupies its own page, collapsing the paging mechanism into a pure transition map. This single-lemma regime enforces strict canon compliance: transitions between attractor states follow the learned structure of the world model, eliminating drift and preserving narrative identity across long trajectories.

A central theme of this paper is the role of the input vector in shaping the attractor flow. Unlike conventional language models, GLPG places no architectural constraints on the composition of its input encoding. The recursive core remains unchanged regardless of how the input vector is constructed, allowing new contextual signals to be added without modifying the underlying architecture. We exploit this flexibility to introduce several enhancements: a normalized line fraction, a high-resolution current sentence representation, multi-scale token histories, and a lightweight narrative memory. These components enrich the temporal and structural information available to the attractor system, improving pacing, segmentation, and stylistic stability.

Beyond input encoding, we refine the training regime through sentence-level sampling, multi-book curricula, structural token annotation, and balanced epoch selection. We further extend the generative engine with a fully parametrizable `WriterParams` layer, enabling explicit control over chapter structure, temporal steering, attractor routing, beam dynamics, and token-level modulation. Optional learned activation and bias functions are explored as modular extensions to the recursive core.

Together, these contributions show that long-form narrative generation does not require large models or probabilistic decoding. A compact attractor system, equipped with structured contextual signals and a parametrizable generation layer, can produce coherent books under strict structural constraints. GLPG illustrates a path toward hybrid intelligence: small deterministic architectures augmented by modular steering layers, capable of evolving beyond the classical “monkeys with typewriters” paradigm.

2 From Paging to Transition Map

In the original GLPG architecture, paging provides a mechanism for grouping multiple input lemmas into shared attractor regions. Each page contains a small set of lemmas, and the recursive dynamics operate within the attractor associated with that page. This allows GLPG to maintain structural coherence while supporting controlled variation inside each page.

For the experiments in this paper, we adopt a simplified variant of GLPG in which each page contains exactly one lemma. In this configuration, the number of pages equals the number of lemmas, and the paging mechanism collapses into a deterministic transition map. Each lemma directly selects its own attractor state, and the model transitions deterministically between these states according to the learned transition structure.

This single-lemma paging regime is no longer a paging system in the traditional sense. Instead, it behaves as a pure transition map: every lemma corresponds to a unique attractor, and the model generates only transitions that are canon-compliant with respect to the underlying world model. This ensures maximal structural stability and eliminates drift between unrelated attractor regions.

The same principle appears in the design of procedural DMLG, where each lemma is associated with its own dynamically generated module. In both cases, the architecture enforces strict lemma-specific transitions, guaranteeing that the generative process remains aligned with the canonical structure of the training corpus.

3 Improving the Input Vector

Although the input vector of a GLPG model is fixed at training time, its internal composition is highly flexible. GLPG places no architectural constraints on how the input vector is constructed, and the recursive core of the model operates identically regardless of the chosen encoding scheme. As a result, different input encoders—potentially combining multiple sources of information—can be designed without requiring any modification to the rest of the architecture.

This flexibility is unusual among language model architectures, where the input representation is typically tightly coupled to the internal layers and difficult to alter without retraining the entire system. In GLPG, the attractor dynamics are robust enough to accommodate a wide range of input encodings, provided that the model is trained with the desired input vector structure from the outset.

Consequently, improvements to the input vector can have a substantial impact on model quality. Richer or more structured input encoders allow GLPG to incorporate additional contextual signals while preserving its compact architecture and deterministic attractor behavior. The following subsections explore several ways in which the input vector can be enhanced to strengthen the expressive capacity of GLPG.

3.1 Book Line Number Fraction

A simple yet highly effective enhancement to the input vector is the inclusion of a *book line number fraction*. During training, every sentence in each source book is assigned a normalized position value in the interval $[0, 1]$, reflecting its sequential location within the book. A sentence at the beginning receives a value near 0, one at the end receives a value near 1, and all intermediate sentences are mapped linearly between these extremes.

During generation, the input vector is augmented with a single float representing the current position in the generated sequence, likewise normalized to $[0, 1]$. The number of generated lines does not need to match the number of lines in any training book; the model simply interprets the fraction as a positional cue within an abstract narrative flow.

This positional signal allows GLPG to infer where a generated line is situated relative to the global structure learned from the training corpus. Even though it consists of only one scalar value, the line fraction provides a clear and continuous encoding of narrative progression, enabling the model to align its output with the structural patterns present in the original books.

3.2 Improving Current Sentence Representation

The current sentence representation provides one of the strongest and most immediately useful signals in the input vector. Unlike the multi-sentence mechanisms (narrative memory, token history), this component focuses exclusively on the structure of the *ongoing* sentence. It captures the most recent grammar and lemma activity, the local punctuation state, and a normalized positional index.

The representation maintains two fixed-length queues: one for grammar tokens and one for lemma tokens. Each queue stores the last N embeddings, where N is a configurable window size (set to 10 in this paper). When a new token arrives, its embedding is inserted at the front of the corresponding queue, and older entries are shifted to the right. This produces a compact, temporally ordered view of the most recent syntactic activity.

Lemma tokens increase the sentence position counter, which is normalized by a divider constant to produce a stable positional feature:

$$\text{pos}_{\text{norm}} = \frac{\text{position}}{\text{divider}}.$$

Terminal tokens (e.g. punctuation) also advance the position counter and set a punctuation flag. Non-terminal grammar tokens update the grammar queue without affecting the position.

The final current-sentence vector is constructed as

$$\text{CurrentSentence} = \text{GrammarQueue} \parallel \text{LemmaQueue} \parallel [\text{punctuation}] \parallel [\text{pos}_{\text{norm}}],$$

where $\parallel$ denotes concatenation. This structure provides the model with a high-resolution snapshot of the ongoing sentence: recent grammar patterns, recent lexical choices, punctuation state, and local progress within the sentence.

Because the representation is updated token-by-token and reflects the immediate syntactic context, it offers a strong stabilizing influence on generation. It helps the model maintain coherent sentence structure, predict appropriate follow-up tokens, and avoid syntactic drift. While the configuration used here (window size $N = 10$ and a single punctuation flag) is effective, it is not necessarily optimal; richer punctuation signals or alternative window sizes may further improve performance.

3.3 Multiplying Token History

The token history mechanism provides a lightweight recurrent trace of recent lemma embeddings. In the baseline configuration, a single history vector of fixed width and decay parameter was used (typically width 48 with decay 0.95). While effective as a stabilizing signal, a single temporal profile limits the model to one characteristic memory scale.

To improve temporal resolution, the system now employs *multiple* parallel token histories. Each history instance maintains its own embedding width and decay coefficient, producing a set of recurrent vectors that capture different temporal dynamics of the recent token stream. The configuration used in this work consists of seven histories:

$$\text{history size} = 8, 16, 32, 48, 64, 80, 96,$$

$$\text{history alpha} = 0.80, 0.85, 0.90, 0.95, 0.97, 0.98, 0.99.$$

Each history vector evolves according to its own decay parameter α , blending new lemma embeddings with its internal state:

$$h \leftarrow \alpha h + (1 - \alpha)e,$$

where e is the current lemma embedding. Smaller widths emphasize short-term fluctuations, while larger widths accumulate broader contextual trends. Lower decay values respond quickly to new tokens, whereas higher decay values retain long-range structure.

The combined history signal is additive rather than substitutive: all history vectors are concatenated into the input vector, providing the model with a multi-scale temporal representation of recent activity. This multi-resolution design allows the generator to exploit both fine-grained and coarse-grained temporal patterns without requiring complex recurrent architectures.

The specific configuration above is chosen for demonstration purposes and is not necessarily optimal. The ideal combination of widths and decay parameters remains an open question and may depend on the curriculum, the narrative domain, or the desired stylistic stability. Nevertheless, multiplying token history significantly enriches the temporal structure of the input vector and improves the expressive capacity of the generator.

3.4 Improving Narrative Memory

The narrative memory mechanism provides a deterministic and lightweight history signal derived from the syntactic-semantic backbone of recent sentences. Because all generated sentences follow a strict $\langle G \rangle \langle L \rangle$ structure—where each grammar token G_i is immediately followed by its corresponding lemma token L_i —the extraction procedure retains all semantically relevant lemma embeddings in their original order.

A fixed set of history tokens marks which grammar categories contribute to the memory:

$\langle \text{VERB-PRESENT} \rangle$, $\langle \text{VERB-PRESENT-3S} \rangle$, $\langle \text{VERB-PRESENT-1S} \rangle$,
 $\langle \text{VERB-PAST} \rangle$, $\langle \text{VERB-ING} \rangle$, $\langle \text{VERB-INGV} \rangle$, $\langle \text{VERB-PERFECT} \rangle$,
 $\langle \text{NOUN} \rangle$, $\langle \text{NOUN-PLURAL} \rangle$, $\langle \text{PROP} \rangle$, $\langle \text{ADJ} \rangle$, $\langle \text{ADV} \rangle$

Whenever such a token is encountered, the next lemma token is guaranteed to be its semantic counterpart. The memory module therefore collects the embeddings of all relevant lemma tokens in the sentence, producing a variable-length sequence that captures the sentence’s syntactic core.

This sequence is processed by a deterministic recurrent projector,

$$h \leftarrow \tanh(W_h h + W_e e + b),$$

followed by a linear projection,

$$P = Vh + c,$$

yielding a compact sentence-level vector. The memory stores the most recent M projections in a temporally ordered structure of size $L \times M$, shifting older entries to the right and inserting the newest projection at position 0.

Although intentionally low-bandwidth, this mechanism provides a stable and reproducible temporal signal. It preserves the sequence of recent actions, entities, and modifiers without attempting deep semantic encoding. Within the broader input vector, narrative memory contributes lightweight contextual coherence, syntactic rhythm, and structural stability across multi-sentence generation.

3.5 The Combined Input Vector

The GLPG input vector consists of 1307 floats. Each component contributes distinct structural, temporal, or variational signals to the attractor dynamics. The vector is composed of four major subsystems: line number, current sentence encoding, multiple token histories, and narrative memory.

Overview

$$1 + 322 + 344 + 640 = 1307 \text{ floats}$$

1. Line Number (1 float) A single normalized scalar indicating the global position of the current sentence within the generated document. This value provides temporal anchoring and supports global pacing, arc progression, and chapter-level flow.

2. Current Sentence Encoding (322 floats) The current sentence is encoded through a combination of grammar embeddings, lemma embeddings, positional information, and punctuation state.

- last 10 grammar embeddings $\times 16$ floats = 160
- last 10 lemma embeddings $\times 16$ floats = 160
- positional scalar (normalized) = 1
- comma / punctuation flag (0 or 1) = 1

Total: 322 floats.

This subsystem provides micro-structure: paragraph formation, chapter boundaries, scene breaks, and local pacing.

3. Multiple Token Histories (344 floats) Seven history windows encode lemma-driven temporal continuity with varying widths and decay factors. Each history window maintains a recurrent trace of previous lemma embeddings.

- history sizes: 8, 16, 32, 48, 64, 80, 96
- history alphas: 0.8, 0.85, 0.9, 0.95, 0.97, 0.98, 0.99

Total: 344 floats.

Token history provides variational modulation: lemma-route diversity, syntactic nuance, micro-pacing variation, and emergent mini-plots.

4. Narrative Memory (640 floats) A recurrent projection of the previous 20 sentences, each represented by a 32-float embedding derived from the main lemma tokens of the sentence.

$$20 \times 32 = 640 \text{ floats}$$

Narrative memory contributes micro-modulation: subtle rhythmic coloration, light smoothing, and small callbacks. It does not carry structure, pacing, canon, or lemma-routes.

Summary The combined input vector integrates structural drivers (line number, current sentence), variational modulators (token histories), and micro-modulation (narrative memory). The attractor core remains responsible for canon, syntax, lemma-routes, scene-flow, and the autonomous narrative motor.

3.6 Ablation Tests

Using the base model, we ablate specific components of the input vector by setting subsets of the 1307 floats to zero. No new models are trained; the only difference between conditions is the removal of encoded information in the input vector. This reveals which components contribute to structure, pacing, variation, and global narrative flow.

Quantitative Overview

- baseline full vector : 1225 s, 258 KB, 128 chapters
- no line number : 1285 s, 260 KB, 127 chapters
- no current sentence : 1217 s, 253 KB, 1 big chapter, no paragraphs
- no token histories : 927 s, 255 KB, 125 chapters
- no narrative memory : 890 s, 255 KB, 126 chapters
- all missing : 844 s, 247 KB, 1 big chapter, no paragraphs
- only current sentence : 986 s, 256 KB, 124 chapters
- only line number + current sentence : 890 s, 256 KB, 128 chapters

Dropping token history or narrative memory significantly increases generation speed. The current sentence encoding is crucial for paragraph and chapter structure. Best speed and stability occur when line number + current sentence are present and narrative memory is absent.

Global Findings The model has an internal narrative motor that continues to operate even when nearly all input information is removed. GLPG continues to generate coherent text under ablations that would break any transformer-based language model, because it is an autonomous attractor system whose narrative dynamics are internally carried.

The transition map and the encoding in the target embeddings form a strong internal steering layer. The context MLP is not a primary motor, but a fine-tuning layer on top of an autonomous attractor.

Role of Line Number Without line number, GLPG loses global flow and remains stuck in a “beginning-of-book” attractor mode. Current sentence provides micro-structure, but line number is essential for arc progression, chapter flow, and temporal anchoring.

Role of Current Sentence Without current sentence, all chapter and paragraph structure disappears. The rest of the narrative remains surprisingly intact, because canon, syntax, lemma-routes, and scene-flow are attractor-internal. Current sentence is the primary structural driver.

Role of Token History Without token history, the narrative remains fully intact, but the flow becomes smoother, less layered, and less variable. Token history is a variation injector, not a structural component. It modulates lemma-route diversity, micro-pacing, and emergent mini-plots.

Role of Narrative Memory Narrative memory remains the least important component. It contributes only micro-modulation: subtle rhythmic coloration, small callbacks, and light smoothing. Canon, syntax, pacing, structure, and lemma-routes are fully attractor-internal.

Minimal Steering Vector The combination “line number + current sentence” is qualitatively almost as strong as baseline. It is far stronger than the extreme ablation and preserves nearly all narrative properties of the baseline model. What is missing: global arc structure, additional lemma variation, and extra pacing nuance. The core—canon, syntax, scene-flow, lemma-routes—remains fully intact.

4 Adding Structural Tokens

To obtain full long-form book structure directly from GLPG generation, two structural terminal grammar tokens are introduced into the decoding process:

- `<SOC>` – Start of Chapter
- `<SOP>` – Start of Paragraph

These tokens act as explicit structural delimiters within the grammar. Both `<SOC>` and `<SOP>` may only appear at the beginning of a sentence, immediately following the `<EOL>` token of the previous line. A paragraph start cannot coincide with a chapter start; therefore, `<SOP>` and `<SOC>` are mutually exclusive on the same line.

Minimum line thresholds enforce structural cadence. A configurable lower bound (e.g. three lines per paragraph and twenty lines per chapter) ensures that structural tokens cannot be emitted prematurely. Rule-based transition constraints prevent the selection of `<SOC>` or `<SOP>` until the required number of lines has elapsed since the previous structural boundary.

With only these two additions, GLPG is able to produce complete book-like documents during autonomous generation. Chapters emerge naturally as long-range units, while paragraphs provide mid-scale segmentation. The narrator pipeline subsequently generates chapter titles and performs structural refinement, but the underlying chapter and paragraph boundaries originate from the grammar-level structural tokens.

5 Improving the Training Regime

Two major refinements were introduced into the GLPG training regime, each aimed at increasing structural stability, narrative coherence, and curriculum diversity across long-form generation.

5.1 Sentence-Level Training

The first improvement replaces sequence-level training with sentence-level training. Each individual line of the book becomes an independent trainable unit, while its contextual embedding is derived from all preceding lines. This approach removes the imbalance between long and short sequences and ensures that every line contributes equally to the learning dynamics.

A continuous *line fraction* is assigned to each sentence, ranging from 0 at the beginning of the book to 1 at the end. This scalar provides the model with a positional signal that captures narrative progression without requiring explicit sequence indices. The model thus learns how early-book and late-book sentences differ in tone, pacing, and structural behavior.

5.2 Training Across Multiple Books

The second improvement expands the curriculum from a single book to multiple books. All books are decomposed into sentences, each annotated with its own line fraction based on its position within its respective volume. This exposes the model to multiple examples of narrative flow from start to finish, greatly increasing the diversity of structural patterns it encounters.

By observing several independent trajectories from line fraction 0 to line fraction 1, the model learns generalized long-range behavior rather than overfitting to the structure of a single text. This multi-book curriculum also improves robustness when generating autonomous drafts, as the model internalizes a broader distribution of narrative arcs.

5.3 Structural Token Annotation

All curriculum sentences are annotated with the structural grammar tokens `<SOC>` and `<SOP>`. These annotations inform the model of valid chapter and paragraph boundaries, enabling it to learn the conditions under which structural tokens appear. During generation, the rule-based transition system enforces minimum line thresholds, while the learned structural patterns guide the placement of chapter and paragraph starts.

5.4 Balanced Epoch Sampling

Each training epoch selects a single random sentence from the curriculum. This uniform sampling strategy ensures that epochs are balanced across the entire dataset, avoiding the uneven weighting that arises when training on long sequences. The model receives a steady stream of diverse structural contexts, improving stability and reducing bias toward any particular region of the training corpus.

Together, these improvements yield a training regime that is more stable, more structurally aware, and better suited for autonomous long-form generation. The resulting models exhibit stronger narrative flow, more consistent structural segmentation, and greater resilience across lineage scales.

6 Fully Parametrizable Generation

A defining characteristic of the GLPG generative engine is its ability to operate under a fully parametrizable configuration layer. Unlike traditional language models, which expose only stochastic sampling controls such as temperature or top- k sampling, GLPG accepts a structured `WriterParams` object that governs narrative structure, temporal progression, attractor routing, and token-level dynamics. This transforms generation from a probabilistic decoding process into a configurable dynamical system.

6.1 Structural Constraints as First-Class Parameters

The parameter class includes explicit structural constraints such as minimum chapter length, minimum paragraph length, sentence word bounds, and maximum chapter counts. These constraints are enforced by the generative engine itself. GLPG’s narrator pipeline integrates these constraints directly into the transition map and lemma routing, ensuring that every generated book respects the requested macro-structure. This stands in contrast to transformer-based models, which cannot guarantee structural compliance and typically rely on prompt heuristics.

6.2 Temporal and Canonical Steering

The parameters `from_line_fraction` and `to_line_fraction` introduce a temporal steering mechanism that selects a specific region of the canon manifold. This allows the generator to operate within a controlled slice of the narrative attractor space. Keywords and prompt lines further act as attractor selectors, influencing lemma clusters and world-model routing rather than merely biasing token probabilities.

6.3 Beam Dynamics and Multi-Route Exploration

Beam-search parameters (`nr_of_beams`, `beam_alpha`, `beam_jitter`, `beam_temperature`) control a multi-route exploration mechanism embedded within the generative engine. In GLPG, beams represent parallel dynamical trajectories through the attractor landscape rather than classical decoding paths. Score damping and jitter introduce controlled stochasticity, enabling exploration without destabilizing the underlying narrative physics. This design allows GLPG to maintain coherence even under extreme parameter settings.

6.4 Token-Level Steering and Vocabulary Modulation

Parameters such as `top_k`, `temperature`, `top_boost`, and `max_tokens` provide fine-grained control over token selection. Unlike classical sampling, these parameters modulate the internal scoring of grammar and lemma tokens produced by the transition map. The generator adjusts the dynamical scoring of candidate transitions rather than sampling from a static distribution. The `max_pageless_vocab` parameter enables a hybrid mode in which the MLP-only generator operates on a random subset of the vocabulary, demonstrating that GLPG can function even when graph-based routing is partially disabled.

6.5 Graph vs. Pageless Generation Modes

The flag `enable_paging` toggles between two fundamentally different generation modes. In graph mode, GLPG uses its full lemma-transition graph, attractor clusters, and world-model routing. In pageless mode, the generator relies solely on the MLP transition map and target embedding encoding. Both modes remain fully functional, and pageless generation continues to produce coherent narrative output. This indicates that the internal attractor dynamics are strong enough to sustain generation even when structural routing is removed.

6.6 A Configurable Dynamical System

Taken together, the `WriterParams` class defines a generative engine that is not merely tunable but architecturally configurable. Each parameter influences a distinct layer of the system: structural constraints, temporal steering, attractor routing, beam dynamics, token scoring, and vocabulary modulation. This level of configurability is unprecedented in traditional language models, where sampling parameters operate only at the surface level. GLPG’s design exposes the internal mechanics of generation, allowing researchers to shape the behavior of the dynamical system itself.

7 Bringing Back Learned Activation Functions

A baseline GLPG model is first trained using the fixed SiLU activation function. This establishes a stable attractor flow and provides a deterministic reference architecture against which subsequent modulation strategies can be evaluated. The fixed activation ensures that the recursive dynamics converge reliably, forming a well-defined attractor landscape that serves as the cognitive core of the model.

To explore whether GLPG can benefit from a learned nonlinearity, a small activation MLP is trained to approximate the SiLU function. This auxiliary network consists of a single input, a single output, and one hidden layer, with SiLU as its internal nonlinearity. Its purpose is not to introduce a new activation regime, but to reproduce the baseline SiLU curve with high fidelity. By doing so, the transition from a fixed activation to a learned activation becomes smooth and controlled, ensuring that the attractor dynamics are not disrupted abruptly.

After this pretraining phase, the activation MLP is inserted into the GLPG architecture as a drop-in replacement for the fixed SiLU function. The model is then trained for additional epochs under this hybrid activation scheme. This yields a GLPG variant with a learned activation function embedded in

its recursive core, analogous to the activation mechanism used in DMLG architectures, but implemented here as a modular plugin rather than a global default.

Experiments were conducted using the learned activation function in different subsets of the hidden layers, as well as across all layers. Interestingly, the learned activation MLP remained almost identical to the pretrained SiLU curve, even after extended training. This suggests that SiLU is already well aligned with the attractor dynamics of GLPG, and that the additional computational cost of a learned global activation function does not provide tangible benefits. The attractor appears to prefer the original SiLU shape, indicating that GLPG’s recursive structure is inherently compatible with this fixed nonlinearity.

8 Introducing an Optional Learned Bias Function

Instead of augmenting the recursive layer with an additional activation MLP, we introduce a lightweight *bias MLP* that learns to inject a context-dependent bias directly into the recursive update. This bias term can be trained as a *global* modulation function, shared across all lemmas, or as a *lemma-specific* function that adapts the recursive dynamics based on the semantic identity of the current token.

The motivation for this design is rooted in the structural properties of GLPG. As a recursive architecture, GLPG exhibits a highly malleable cognitive substrate: its attractor dynamics are stable enough to preserve global structure, yet flexible enough to accept fine-grained modulation without destabilizing the overall flow. A learned bias function exploits this plasticity by altering the local curvature of the vector field that governs the recursive update. Rather than replacing the underlying dynamics, the bias MLP subtly reshapes them, allowing the model to express richer contextual variation while maintaining canonical consistency.

Bias MLPs are computationally efficient, stable under training, and provide qualitatively meaningful modulation. They operate as a lightweight hypernetwork: instead of generating weights, they generate a bias vector that shifts the recursive update in a context-sensitive direction. This enables GLPG to adapt its internal state transitions without increasing model size or compromising the stability of the attractor.

8.1 Bias-Conditioned Dynamics in the Full Forward Pass

The forward computation of GLPG consists of an initial projection, followed by a sequence of recursive gated updates and a final refinement stage. The learned bias function integrates specifically into the recursive block, where it modulates the attractor dynamics.

Let the input be denoted by x . The model first computes an internal state

$$h_0 = \text{silu}(W_1 x), \quad h_1 = \text{silu}(W_2 h_0),$$

which serves as the initial condition for the recursive process.

Recursive gated update. Each recursive step applies a gated residual transformation. For iteration t , the gate value is computed as

$$g_t = \sigma(Gh_t),$$

with $g_t \in (0, 1)$ controlling the effective depth of the recurrence.

The default attractor update is given by

$$u_t^{(0)} = \text{silu}(Ah_t),$$

where A defines the baseline vector field of the recursive dynamics.

When the bias function is enabled, a context-dependent modulation term

$$b_t = B(h_t)$$

is added to the attractor update, yielding

$$u_t = \text{silu}(Ah_t + b_t).$$

The recursive state transition then becomes

$$h_{t+1} = h_t + g_t(u_t - h_t),$$

which shows that the bias term directly influences the direction and magnitude of the recursive update. Because this modulation occurs inside the recurrence, it affects multiple successive transitions, allowing the attractor trajectory to adapt dynamically to the context encoded in h_t .

Refinement stage. After the recursive block, the model applies a final refinement transformation

$$r = \text{silu}(Rh_T), \quad h_{\text{final}} = h_T + r,$$

which stabilizes the attractor and ensures that the modulation introduced by the bias does not destabilize the global structure of the state space.

Output. The final output is produced by a linear projection

$$y = Oh_{\text{final}}.$$

Interpretation. This formulation reveals that the bias function acts as a lightweight hypernetwork: it does not generate weights, but instead produces a context-sensitive perturbation of the vector field governing the recursive update. The attractor defined by A remains intact, while the bias term b_t reshapes its local curvature. This enables GLPG to express richer contextual variation without increasing model size or compromising stability.

9 Testing the Effect of the Bias Function

This section evaluates how a learnable bias function influences a GLPG model trained entirely from scratch. The experiment examines the formation of a bias-conditioned manifold, the internalization of the bias field, and the effect of scaling the bias during inference through an adjustable guidance coefficient α .

9.1 Training Sequence

The model was trained in sequential phases:

- Hidden size: 768.
- Warmup without bias (1K epochs): 118 s, 17 884 transitions, model size 54.9 MB.
- Generation test (300 sentences): 155 s, one long chapter.
- Addition of bias-MLP with hidden size: 256.
- 2K bias-MLP initialization epochs (zero-bias): model size 56.4 MB.
- Additional 1K epochs: 140 s, 36 363 transitions, model size 58.5 MB.
- Generation test (300 sentences): 146 s, one long chapter.
- Additional 100K epochs: 2 461 s, 1 883 847 transitions, model size 68.3 MB.

- Generation test (3 000 sentences): 983 s, 245 KB, 127 chapters.
- Additional 200K epochs: 7 904 s, 5 562 998 transitions, model size 68.3 MB.
- Generation test (3 000 sentences): 1 192 s, 258 KB, 122 chapters.

After 100K epochs the model remains partially procedural and fragmentary. After 300K epochs the model exhibits a fully developed bias–canon: a coherent attractor family shaped by the bias–MLP.

9.2 Bias Ablation

To measure how much of the bias field is internalized, generation was performed with the bias disabled:

- Generation with bias disabled ($\alpha = 0$): 1 256 s, 260 KB, 123 chapters.

The output remains canon–consistent but becomes looser, noisier, and less structurally stable. This confirms that the bias–MLP acts as a training–time regularizer: it shapes the manifold and stabilizes attractor dynamics, and part of this structure persists even when the bias is removed.

9.3 Bias Scaling

During inference the bias vector was scaled:

$$b = \alpha \cdot \text{bias}(h), \quad u = \text{silu}(Wh + b).$$

Generation results:

- Bias scaled by $\alpha = 3$: 1 016 s, 258 KB, 123 chapters.
- Bias scaled by $\alpha = 5$: 1 116 s, 264 KB, 125 chapters.
- Bias scaled by $\alpha = 1000$: 1 097 s, 260 KB, 124 chapters.

These ablations demonstrate that the bias–MLP is a learnable modulation field with an adjustable guidance coefficient α , shaping the latent manifold and controlling attractor stability.

9.4 Effect of the Guidance Coefficient

The following regimes were observed:

- $\alpha = 0$: free, noisy, reduced canon stability.
- $\alpha = 1$: organic, emergent, high variation.
- $\alpha = 3$: sweet spot; canon–stable, smooth pacing, coherent scene flow.
- $\alpha = 5$: strongly guided; rigid, hyper–canon, reduced variation.
- $\alpha = 1000$: extreme guidance; the bias field fully dominates the recurrent update, producing a single, monolithic attractor with highly stable but minimally varied output.

At $\alpha = 1000$ the recurrent update becomes numerically dominated by the scaled bias term:

$$u \approx \text{silu}(1000 \cdot \text{bias}(h)),$$

rendering the contribution of Wh negligible. As a result, the bias–MLP becomes effectively self–recursive: its output reenters the bias network at each timestep, forming a closed procedural loop. This produces a deterministic bias–attractor characterized by:

- highly stable syntactic rhythm,
- fixed lemma–routing patterns,
- repeated canon clusters,
- uniform pacing across chapters,
- minimal emergent variation.

Despite the extreme scaling, the model remains fully functional and continues to generate coherent text. This robustness indicates that the GLPG substrate tolerates large perturbations in its modulation field without collapsing, behaving as a dynamical system whose attractor geometry can be overwritten by sufficiently strong guidance.

The $\alpha = 1000$ regime therefore reveals the limiting behaviour of the bias–MLP: a pure procedural attractor driven entirely by the modulation field, demonstrating that the bias function can assume complete control over the recurrent dynamics when scaled beyond the influence of the main update term.

9.5 Relation to Known Techniques

The mechanism resembles several known forms of guided training:

Auxiliary Networks. A small network shapes the manifold, reduces noise, stabilizes gradients, and prevents attractor chaos. During inference the auxiliary network may be disabled while its structural imprint remains.

Teacher Forcing and Scheduled Sampling. The model learns under structured guidance but generates without it. The learned manifold remains smoother and more coherent than models trained without guidance.

Unlike Transformers, GLPG possesses a recurrent attractor state and lemma–conditioned manifold, making α –scaled modulation dynamically stable and architecturally feasible. The bias–MLP therefore functions as a genuine learnable guidance engine within the GLPG substrate.

10 Evolving to Fully Procedural GLPG

Procedural GLPG refers to architectures in which parts of the network are generated *on demand* rather than stored as fixed parameters. The recursive structure of GLPG naturally supports such procedurality: its attractor dynamics are stable enough to tolerate modulation, yet flexible enough to benefit from context-dependent variation. In the previous section, we introduced a lightweight form of procedural modulation through a learned bias function applied inside the recursive update. This constitutes the simplest procedural extension of GLPG. We now outline how this idea can be expanded into a fully procedural architecture in which entire hidden layers are generated dynamically.

10.1 Lightweight Procedural GLPG

The bias-conditioned recurrence described earlier already provides a minimal form of procedurality. The modulation term is computed at runtime from the current hidden state, allowing the recursive flow to adapt dynamically without altering the stored parameters of the network. This approach preserves the compactness of GLPG while introducing a controlled degree of procedural behavior. It serves as a practical first step toward more advanced forms of procedural GLPG.

10.2 Full Procedural GLPG: Dynamic Low-Rank Layer Generation

A fully procedural GLPG replaces its internal hidden layers with dynamically generated parameters. Rather than storing dense weight matrices, the model uses lemma-conditioned hypernetworks to produce low-rank factorizations of these matrices at runtime. This approach preserves the expressive power of GLPG while avoiding the prohibitive cost of generating full parameter sets for large hidden sizes.

To make the discussion concrete, consider a GLPG model with hidden size 256. A dense layer of this size contains

$$256 \times 256 + 256 = 65,792$$

parameters. Generating such a matrix procedurally for every lemma would be computationally infeasible. Instead, each hidden layer is represented through a rank-16 factorization:

$$W = AB,$$

where $A \in \mathbb{R}^{256 \times 16}$ and $B \in \mathbb{R}^{16 \times 256}$. The total number of parameters per layer becomes

$$256 \cdot 16 + 16 \cdot 256 = 8,192,$$

a reduction by nearly an order of magnitude. This low-rank representation is sufficient to modulate the attractor dynamics while keeping the procedural generators compact and trainable.

GLPG contains three internal hidden layers and one gate layer. In the fully procedural variant, each of these four layers is replaced by a dedicated lemma-conditioned generator MLP that outputs the corresponding low-rank factors (A, B) . A fifth generator produces the bias vector used inside the recursive update. Together, these five procedural modules define the lemma-specific dynamics of the network.

Training. During training, the procedural generators are updated through backpropagation exactly like ordinary network components. The generated parameters are transient: they exist only during the forward pass. Gradients flow through the generated low-rank factors into the generator MLPs, allowing them to learn how to produce effective layer parameters without ever storing the full matrices. This approach scales well even for large lemma vocabularies, since the procedural generators remain small and shared across all lemmas.

Inference. At inference time, the procedural generators compute the low-rank factors (A, B) for each layer based on the current lemma embedding and hidden state. The resulting matrices are constructed on the fly and used to evaluate the GLPG recurrence. Caching is optional but not required. The model behaves identically to a standard GLPG except that its internal dynamics are now lemma-conditioned and context-sensitive.

Memory footprint. The total footprint of fully procedural GLPG consists of:

- all learned target embeddings (128-dimensional),
- the shared input and output projection layers,
- four low-rank procedural generators (for three hidden layers and one gate layer),
- one bias generator for lightweight modulation.

For a lemma space of size 100,000, storing dense parameters for four layers would require

$$100,000 \times 65,792 = 6.5\text{G parameters},$$

which is impractical. The procedural low-rank approach represents all of these parameters implicitly through compact generator networks, trading memory for compute. Since inference bottlenecks are typically dominated by memory bandwidth rather than arithmetic throughput, this trade-off is advantageous.

Summary. Lightweight procedural GLPG introduces dynamic modulation through a learned bias function, while fully procedural GLPG extends this idea by generating entire hidden layers through lemma-conditioned low-rank hypernetworks. This architecture enables fine-grained control over attractor dynamics, supports lineage and modularity, and preserves the compactness of GLPG. The low-rank formulation makes full procedurality feasible even for moderately sized hidden layers, providing a practical path toward highly expressive and context-adaptive GLPG models.

11 The Four Gutenberg Books

These four public-domain novels form the superposed canon used to construct the GLPG world model. Together they provide a diverse mixture of science fiction, planetary romance, Victorian horror, and early speculative literature. Their combined narrative space defines the semantic substrate from which GLPG learns its attractor dynamics.

Book	Sentences
<i>A Honeymoon in Space</i>	3797
<i>Thuvia, Maid of Mars</i>	2841
<i>The Time Machine</i>	2094
<i>Dr. Jekyll and Mr. Hyde</i>	1809

Table 1: Overview of the four Gutenberg books forming the GLPG superposed canon.

Across these four works, the combined corpus contains a total of

10541 sentences

from which

194085 unique training samples

were extracted. The resulting vocabulary comprises

8562 distinct lemmas,

forming the lexical and conceptual foundation of the GLPG world model.

12 Behavior of GLPG at Different Lineage Scales

The GLPG architecture exhibits distinct behavioral regimes depending on lineage depth, hidden-state scale, and accumulated training epochs. Each lineage variant is trained as an independent cognitive substrate, inheriting only the compressed transition statistics of its parent when applicable. The table below summarizes the principal configurations and their empirical characteristics during training and autonomous draft-book generation.

For very small models, the embedding matrices dominate the serialized footprint; their size remains constant across all variants. Despite GLPG’s non-transformer architecture, the lineage exhibits a clear cognitive scaling curve analogous to LLM scaling laws. This behavior provides predictive evidence for the cognitive potential of procedural GLPG systems: larger hidden states and deeper lineage produce disproportionately richer generative dynamics, even with minimal additional epochs.

Class	Model Size	Hidden	Parent	Add. Epochs	Transitions	Train (s)	Gen (s)	KB	Ch.
base	92 MB	1024	none	300000	5505880	12344	1087	257	124
base+bias	92 MB	1024+8	base	100000	7346679	5629	1127	266	127
bias	68 MB	768+256	none	302000	5562998	10365	1192	258	123
large	252 MB	2048	base	200000	9199949	33724	1195	263	127
xl	862 MB	4096	large	40000	9938266	23244	1629	267	126
xxl	3.15 GB	8192	xl	2000	9975674	4537	4206	261	125
small	48 MB	512	large	200000	12886275	3204	1053	265	127
tiny	34 MB	256	small	200000	16565029	1985	1063	268	132
micro	28 MB	64	tiny	200000	20243999	2108	875	265	136
xxl micro	3.15 GB	8192	micro	2000	20280933	1746	3808	270	137

Lineage inheritance plays a decisive role. The XXL model, when trained on top of XL parent, converges rapidly and exhibits stable long-range behavior after only a few thousand additional epochs. Conversely, the micro-based XXL lineage benefits from its vastly larger accumulated transition count, producing more structurally diverse outputs despite identical hidden-state scale.

All lineage variants are capable of generating fully autonomous draft books. Each book is distinct yet consistently situated within the superposed canon world derived from the four original training novels. Draft generation typically spans 3000 lines, with a configurable line-fraction from 0 (start) to 1 (end). A minimum of 20 lines per chapter yields a theoretical maximum of 150 chapters. Generation time and output size vary per run, as the models produce highly divergent narratives from scratch, guided only by the initial random seed.

13 The New Narrator Pipeline

The narrator pipeline introduces a second generative stage that operates on top of the raw GLPG output. Instead of producing a finished book directly, GLPG now generates a draft file containing unmoderated chapters, each composed of structurally valid but semantically rough sentences. This draft serves as the substrate for a fully agentic, LLM-driven narrator that transforms the raw material into a coherent, publishable LaTeX book.

13.1 From GLPG Draft to Narrator Input

The first stage consists of pure GLPG sampling. The generator produces a sequence of chapters, each marked by structural boundaries and containing the canonical lemma clusters characteristic of GLPG’s attractor dynamics. These chapters are written to disk as a plain text file. No moderation, scoring, or narrative shaping occurs at this stage; the output is intentionally raw, preserving the full dynamical behavior of the generative engine.

The narrator pipeline reads this file line by line, reconstructing chapter boundaries and extracting the original sentences. Each chapter becomes a `TriadicNarratorChapter` object, containing the unprocessed GLPG output that will be refined in subsequent steps.

13.2 Agentic Moderation and Literary Refinement

The second stage is a purely LLM-driven transformation. Using a dedicated moderation prompt, the narrator applies grammatical correction, semantic repair, narrative arc construction, and descriptive enhancement. The moderation process is iterative: the narrator attempts multiple refinements until the chapter meets minimum structural and semantic criteria. Broken sentences are repaired, gaps are filled, and the chapter is reshaped into a coherent literary unit while remaining faithful to the original GLPG content.

This moderation step is not a simple rewrite. It is an agentic process in which the LLM evaluates its own output, retries failed attempts, and ensures that the final chapter satisfies constraints such as minimum length, narrative coherence, and stylistic consistency. The result is a polished chapter that retains GLPG’s canonical world-model while achieving the readability of a professionally edited text.

13.3 Title Generation and Chapter Scoring

Once a chapter is moderated, the narrator invokes a title-generation prompt to produce a suitable chapter title. The LLM evaluates the chapter’s thematic content and proposes a concise, stylistically appropriate title. If the title is unsatisfactory, the narrator retries until a non-empty, meaningful title is obtained.

The narrator then assigns a numerical score to the chapter using a dedicated scoring prompt. The score reflects coherence, readability, originality, creativity, narrative progression, and suitability as a chapter in a real book. Chapters that fail to meet the minimum score threshold are retried, ensuring that only high-quality moderated chapters appear in the final output.

13.4 Inter-Chapter Transitions

Between consecutive chapters, the narrator tries to generate a short optional transition using a specialized bridging prompt. These transitions do not summarize the chapters; instead, they provide subtle, mood-based connective tissue that enhances the flow of the book. The narrator ensures that transitions introduce no new characters, objects, or events, and that they avoid repetition or collapse. A dedicated repetition detector verifies that the transition is structurally clean before it is accepted.

The result is a seamless narrative progression across chapters, transforming GLPG’s independent outputs into a unified literary work.

13.5 LaTeX Assembly and Final Book Construction

After moderation, titling, scoring, and transition generation, the narrator writes the final book to a LaTeX file. Each chapter is emitted with a proper `\chapter{}` command, followed by the moderated sentences. Transitions are inserted as comment-marked sections, preserving structure while maintaining readability.

The output file is immediately usable as a complete book manuscript. It contains:

- polished chapters with coherent narrative arcs,
- automatically generated titles,
- validated transitions,
- structural metadata such as chapter scores,
- and a fully assembled LaTeX layout.

13.6 Hybrid GLPG × LLM Architecture

This two-stage architecture demonstrates that a compact model such as Mistral-7B can achieve output quality normally associated with much larger LLMs. GLPG provides structural stability, canonical consistency, and attractor-driven world modeling, while the narrator LLM provides semantic refinement, literary shaping, and agentic decision-making.

The result is a hybrid system in which architectural design compensates for model scale. The narrator pipeline elevates raw GLPG output into a fully publishable book, illustrating that generative quality can emerge from the interaction between a deterministic grammar engine and an agentic refinement layer rather than from sheer parameter count alone.

14 Example Generated Books

The narrator pipeline produces complete long-form books through autonomous draft generation, remastering, and final LaTeX rendering. Below is an overview of several example outputs created with different model lineages, draft sizes, and narrator configurations. Each entry represents a fully generated book produced end-to-end by the pipeline.

Model	Draft (KB)	Narrator (KB)	Seconds	Title
base	258	574	23130	Veils of the Red Twilight
base+bias	267	573	21650	Astral Tides of the Hidden Suns
bias	257	542	16352	Astral Gate: Chronicles of the Crimson Arrival
large	261	568	15887	Echoes Across the Crimson Gate
large	263	571	17761	The Ocean Between Suns
xl	267	555	17587	The Martian Twilight
xxl micro	270	578	16239	The Temple of the Mountain Gods

A fully autonomously created omnibus, *The Autonomous Parrot: Martian Pulp Fiction Omnibus*, is available within the ecosystem and contains all seven generated books. This combined work exceeds 1900 pages and demonstrates the pipeline’s ability to produce large-scale, multi-world narrative artifacts.

Important note. These books serve primarily as demonstrations of the current capabilities of the narrator pipeline. The draft-generation stage does not yet enforce long-range coherence across an entire book, and therefore these works should be regarded as technical end-to-end AI artifacts rather than publishable novels. They illustrate the architecture’s autonomy, lineage variation, and world-blending behavior within the ecosystem.

15 Analogy to the Monkeys with Typewriters

The classical “monkeys with typewriters” parable imagines an infinite ensemble of random actors who, given unbounded time, eventually reproduce the works of Shakespeare. Its yield is conceptually nonzero, yet practically negligible: coherence is accidental, structure is absent, and semantic consistency is never guaranteed.

This system, by contrast, operates in a fundamentally different regime. From a single canon source (GLPG) and a single reconstruction engine (LLM), the pipeline produces a quasi-infinite family of canon-consistent pulp books. Variation is not random but procedurally modulated; structure is not emergent by chance but enforced by attractor dynamics; coherence is not a statistical accident but a consequence of hybrid intelligence.

Where monkeys with typewriters require infinite time to stumble upon a single coherent work, the GLP–Graph Book Authoring Pipeline generates a new unique book on every run. The yield is not astronomically small, but astronomically large: a controlled explosion of structured narrative diversity.

In this sense, the pipeline evolves *beyond* the classical analogy. It replaces randomness with guided variation, replaces noise with canon, and replaces chance with architectural intent. This transformation—from stochastic typing to hybrid, procedural authorship—captures the spirit of this paper:

Hybrid Intelligence Evolving Beyond the Monkeys with Typewriters.

16 Conclusion

This work demonstrates that long-form narrative generation does not require large-scale probabilistic models or transformer-based sequence architectures. The GLP–Graph Book Authoring Pipeline shows that coherent books, complete with chapters, paragraphs, pacing, and canon, can emerge from a compact deterministic attractor system when equipped with structured contextual signals and a robust transition map.

By collapsing paging into a single-lemma transition regime, GLPG achieves strict canon compliance and stable attractor routing. The improvements introduced in this paper—richer input encodings, multi-scale temporal histories, a high-resolution current sentence representation, lightweight narrative memory, and normalized line fraction—demonstrate that the expressive capacity of the attractor can be significantly enhanced without increasing architectural complexity. These signals strengthen pacing, segmentation, and stylistic stability while preserving the deterministic nature of the recursive core.

The refined training regime further contributes to structural coherence. Sentence-level sampling, multi-book curricula, structural token annotation, and balanced epoch selection collectively produce a model that generalizes across narrative arcs and maintains stability over long trajectories. The introduction of a fully parametrizable `WriterParams` layer transforms GLPG into a configurable dynamical system, enabling explicit control over macro-structure, temporal steering, attractor routing, beam dynamics, and token-level modulation. Optional learned activation and bias functions illustrate how modular extensions can be integrated without disrupting the underlying attractor physics.

Taken together, these contributions reveal a broader insight: hybrid intelligence can emerge from small deterministic systems when their internal dynamics are enriched with structured contextual signals and exposed steering layers. GLPG provides a concrete demonstration of this principle, offering a compact, interpretable, and parametrizable alternative to conventional language models. Its attractor-based architecture suggests new directions for narrative generation research, where dynamical systems, modular encodings, and explicit structural constraints work together to produce coherent, long-form text far beyond the classical “monkeys with typewriters” paradigm.

17 Triadic Cosmos Ecosystem Context

The present paper is part of the broader *Triadic Cosmos* ecosystem: a unified research program that develops a compositional ontology of intelligence together with modular, inspectable computational architectures. The ecosystem spans conceptual theory, executable reference implementations, and fully documented empirical demonstrations, forming a coherent framework for studying hybrid intelligent systems.

All materials, extended notes, and evolving documentation are publicly available in the open GitHub project:

<https://github.com/triadic-cosmos>

Within the `triadic-system` repository, a complete reference implementation of the DMLG and GLPG engines, the narrator pipeline, and all supporting modules is provided under the AGPL-3.0 license.

References

- [1] J. De Witte, “The GLP–Graph Architecture: Combining GLP–Mono with DMLG,” Triadic Cosmos Ecosystem, 2026.
- [2] J. De Witte, “The GLP–Mono Architecture,” Triadic Cosmos Ecosystem, 2026.
- [3] J. De Witte, “A Narrator Pipeline driven by DMLG: Structural Hybrid Intelligence,” Triadic Cosmos Ecosystem, 2026.
- [4] J. De Witte, “Language as Graph Traversal: The DMLG Architecture,” Triadic Cosmos Ecosystem, 2026.
- [5] J. De Witte, “The Procedural DMLG Architecture: A Large Mind Creating Many Smaller Minds,” Triadic Cosmos Ecosystem, 2026.